Base de datos para un Gimnasio

Introducción

La base de datos está orientada a la gestión de un gimnasio especializado en entrenamiento de pesas y boxeo.

> Permite administrar clientes, entrenadores, membresías, rutinas de pesas, sesiones de boxeo, pagos y asistencias.
>
> El **objetivo** es centralizar la información y preparar la base para una futura página web del gimnasio.

Situación Problemática

El gimnasio actualmente no cuenta con un sistema digital ni página web. Esto genera dificultades para:

- Controlar pagos y vencimientos de membresías
- Registrar asistencia a entrenamientos.
- Llevar un historial de rutinas de pesas y sesiones de boxeo

Esta base de datos propuesta soluciona estas brechas al integrar todo en un único esquema relacional.

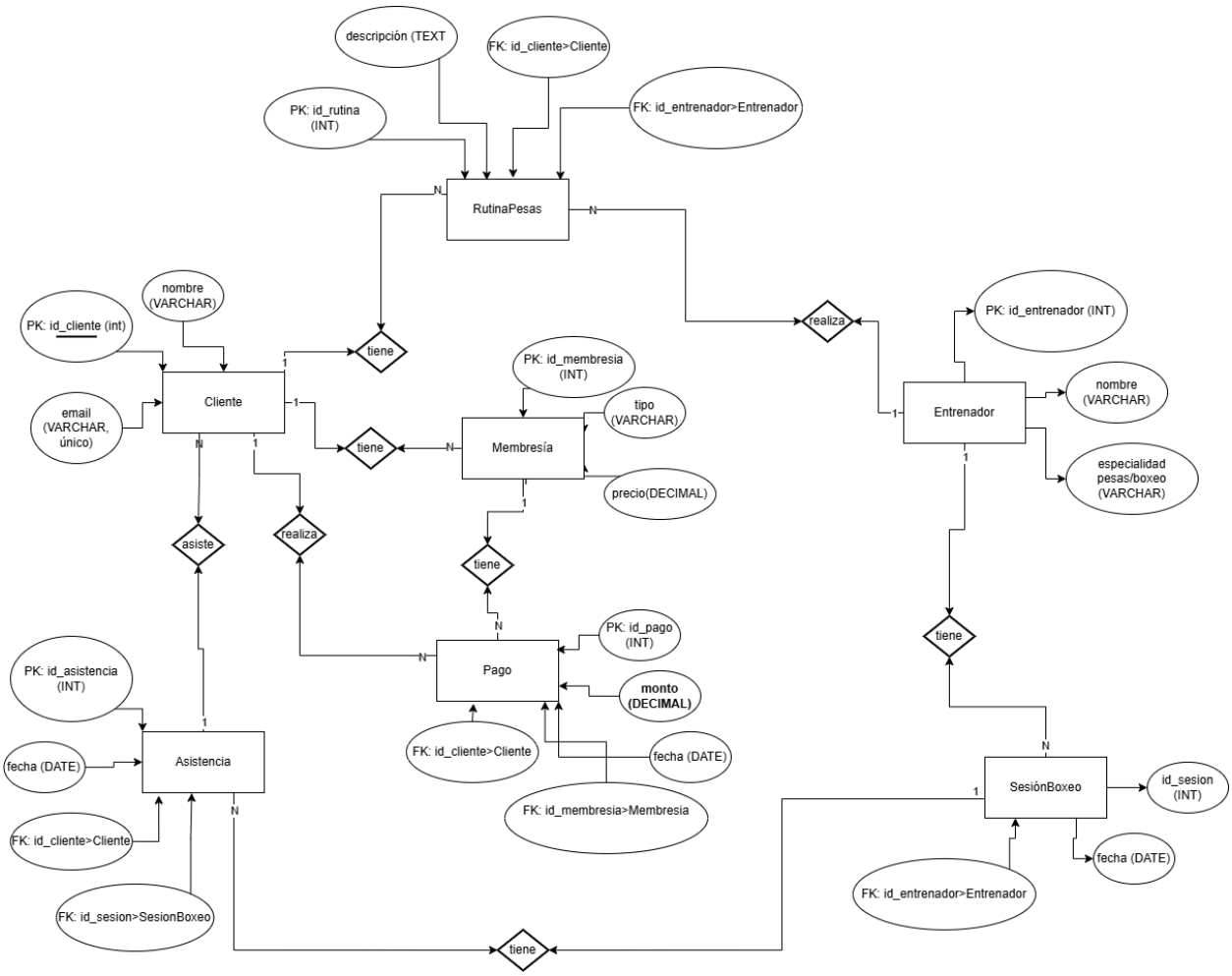
Modelo de Negocio

El gimnasio ofrece:

- Entrenamiento de pesas (con rutinas personalizadas).
- Boxeo (sesiones individuales o grupales reducidas)
- Entrenadores personales para pesas y boxeo.
- Membresías diarias/quincenal/mensuales

El modelo de negocio se centra en la relación:
Cliente ↔ Membresía ↔ Pago ↔ Entrenador/Rutinas ↔ Sesiones de Boxeo ↔ Asistencia.

Diagrama Entidad Relación



Listado de Tablas

TABLA	DESCRIPCION	CAMPOS	PK/FK
Cliente	Datos de clientes	Id_cliente, nombre, email	PK id_cliente
Entrenador	Datos de entrenadores	Id_entrenador, nombre, especialidad	PK id_entrenador
Membresía	Tipo membresía (día/quincenal/mensual)	Id_membresia, tipo, precio	PK id_membresia
RutinaPesas	Rutinas personalizadas	id_rutina, descripcion, id_cliente, id_entrenador	PK id_rutina, FK id_cliente, FK id_entrenador
SesionBoxeo	Sesiones boxeo	id_sesion, fecha, id_entrenador	PK id_sesion, FK id_entrenador

Pago	Registro de pagos	id_pago, fecha, monto, id_cliente, id_membresia	PK id_pago, FK id_cliente, FK id_membresia
Asistencia	Registro de asistencia a clase de boxeo	id_asistencia, fecha, id_cliente, id_sesion	PK id_asistencia, FK id_cliente, FK id_sesion

Script SQL de creación

```
/*create database GimnasioDB;
use GimnasioDB;*/

create table cliente( id_cliente int auto_increment primary key, nombre varchar(100) not null, email varchar(150) unique not null );

create table entrenador (id_entrenador int auto_increment primary key, nombre varchar(100) not null, especialidad varchar(50) not null);

create table membresia ( id_membresia int auto_increment primary key, tipo varchar(50) not null, precio decimal(10,2) not null);

create table rutinapesas (id_rutina int auto_increment primary key, descripcion text not null, id_cliente int,id_entrenador int,
foreign key(id_cliente) references cliente(id_cliente),
foreign key( id_entrenador) references entrenador(id_entrenador));

create table sesionboxeo (id_sesion int auto_increment primary key, fecha date not null, id_entrenador int,
foreign key (id_entrenador) references entrenador(id_entrenador));

create table pago( id_pago int auto_increment primary key, fecha date not null, id_cliente int, id_membresia int,
foreign key(id_cliente) references cliente(id_cliente),
foreign key(id_membresia) references membresia(id_membresia));

create table asistencia (id_asistencia int auto_increment primary key, fecha date not null, id_cliente int, id_sesion int,
```

```
foreign key(id_cliente) references cliente(id_cliente),
foreign key(id_sesion) references sesionboxeo(id_sesion));
```

Continuación – Entrega 2

A continuación, se incorporan nuevos objetos de la base de datos que permiten enriquecer la funcionalidad y preparar el sistema para consultas más avanzadas y automatización de procesos:

- 5 **Vistas** que integran información de clientes, entrenadores, membresías, pagos, rutinas y asistencias, facilitando reportes y consultas rápidas.
- 2 **Funciones** personalizadas que calculan totales de pagos y cantidad de asistencias por cliente, útiles para análisis financiero y de participación.
- 2 **Procedimientos almacenados** que automatizan la inserción de rutinas y pagos, simplificando la gestión operativa del gimnasio.
- 1 **Trigger** que demuestra cómo se pueden automatizar acciones derivadas de eventos, registrando asistencias automáticamente al insertar un pago.
- Scripts .SQL separados para la creación de objetos y la inserción de datos de prueba, verificando que las vistas, funciones y procedimientos puedan ejecutarse correctamente.

Con esta ampliación, la base de datos no solo centraliza la información del gimnasio, sino que también ofrece herramientas para la gestión diaria, el control de pagos y asistencias, y la preparación de reportes que servirán como base para una futura página web o sistema de administración digital.

VISTAS

Vista: vista_clientes_membresia

- Descripción: Muestra clientes junto con el tipo de membresía que poseen.
- Objetivo: Consultar rápidamente qué clientes tienen cada plan.
- Tablas: cliente, membresia, pago

```
create view vista_clientes_membresias as select c.nombre,c.email,m.tipo,m.precio from cliente c
join pago p on c.id_cliente = p.id_cliente
join membresia m on p.id_membresia = m.id_membresia;
```

```
1 •  SELECT * FROM gimnasiodb.vista_clientes_membresias;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

nombre	email	tipo	precio
Juan Perez	juanperez@gmail.com	Mensual	5000.00
Carlos Díaz	carlosdiaz@gmail.com	Mensual	5000.00
Juan Perez	juanperez@gmail.com	Mensual	5000.00
María López	marialopez@gmail.com	Anual	45000.00

1-Vista clientes membresias

Vista: vista_rutinas_clientes

- Descripción: Lista las rutinas de pesas con el cliente y entrenador asignado.
- Objetivo: Facilitar el seguimiento de rutinas personalizadas.
- Tablas: rutinapesas, cliente, entrenador

create view *vista_rutinas_clientes* ***as select*** *r.descripcion,c.nombre* ***as*** *cliente, e.nombre* ***as*** *entrenador* ***from*** *rutinapesas r*

join *cliente c* ***on*** *r.id_cliente = c.id_cliente*

join *entrenador e* ***on*** *r.id_entrenador = e.id_entrenador;*

```
1 •  SELECT * FROM gimnasiodb.vista_rutinas_clientes;
```

Result Grid | Filter Rows: | Export: | Wrap Cel

descripcion	cliente	entrenador
Rutina fuerza nivel 1	Juan Perez	Luis Fernández
Rutina resistencia nivel 2	María López	Luis Fernández
Rutina fuerza nivel 3	Juan Perez	Luis Fernández

2-Vista rutinas clientes

Vista: vista_asistencias_boxeo

- **Descripción:** Muestra las asistencias de clientes a sesiones de boxeo.

- **Objetivo:** Controlar la participación en clases.
- **Tablas:** asistencia, cliente, sesionboxeo

```
create view vista_asistencias_boxeo as select a.fecha, c.nombre as cliente, s.fecha as sesion from asistencia a
join cliente c on a.id_cliente = c.id_cliente
join sesionboxeo s on a.id_Sesion = s.id_sesion;
```

```
1 •   SELECT * FROM gimnasiodb.vista_asistencias_boxeo;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

fecha	cliente	sesion
2026-04-03	Juan Perez	2026-04-03
2026-04-05	Carlos Díaz	2026-04-03
2026-04-06	Juan Perez	2026-04-03
2026-04-04	María López	2026-04-04

3-Vista asistencias boxeo

Vista: vista_pagos_clientes

- **Descripción:** Lista los pagos realizados por cada cliente.
- **Objetivo:** Generar reportes financieros.
- **Tablas:** pago, cliente, membresia

```
create view vista_pagos_cliente as select c.nombre, p.fecha, m.tipo, m.precio from pago p
join cliente c on p.id_cliente = c.id_cliente
join membresia m on p.id_membresia = m.id_membresia;
```

```
1 •   SELECT * FROM gimnasiodb.vista_pagos_cliente;
```

Result Grid | Filter Rows: | Export: | Wra

nombre	fecha	tipo	precio
Juan Perez	2026-04-01	Mensual	5000.00
Carlos Díaz	2026-04-05	Mensual	5000.00
Juan Perez	2026-04-06	Mensual	5000.00
María López	2026-04-02	Anual	45000.00

4- Vista pagos clientes

Vista: vista_entrenadores_especialidad

- Descripción: Muestra entrenadores y sus especialidades.
- Objetivo: Consultar disponibilidad de entrenadores por área.
- Tablas: entrenador

create view *vista_entrenadores_especialidades* ***as*** *select nombre,especialidad* ***from*** *entrenador;*

```
1 •   SELECT * FROM gimnasiodb.vista_entrenadores_especialidades;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nombre	especialidad
Ana Torres	Boxeo
Luis Fernández	Pesas

5-Vista entrenadores especialidades

FUNCIONES

Función: fn_totalPagosCliente

- Descripción: Calcula el total abonado por un cliente.
- Objetivo: Usar en reportes financieros.
- Tablas: pago, membresia

delimiter //

```
create function fn_totalPagosCliente(idCliente int) returns decimal(10,2)
deterministic
begin
declare total decimal(10,2);
select sum(m.precio) into total from pago p
join membresia m on p.id_membresia = m.id_membresia where p.id_cliente = idCliente;
return total;
end;
```

```
52 •  select fn_totalPagosCliente(1) as TotalPagosJuan;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

	TotalPagosJuan
▶	10000.00

6-Funcion total pagos clientes

Función: fn_cantidadAsistenciasCliente

- Descripción: Devuelve la cantidad de asistencias de un cliente.
- Objetivo: Evaluar participación en actividades.
- Tablas: asistencia

```
delimiter //
create function fn_cantidadAsistenciasCliente(idCliente int) returns int
deterministic
begin
declare cantidad int;
select count(*) into cantidad from asistencia where id_cliente = idCliente;
return cantidad;
end;
```

```
53 •  select fn_cantidadAsistenciasCliente(2) as AsistenciasMaria;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

	AsistenciasMaria
▶	1

7-Funcion cantidad asistencias cliente

Stored Procedures

SP: sp_registrarRutinaPesas

- Descripción: Inserta una nueva rutina asociada a un cliente y entrenador.
- Objetivo: Automatizar la carga de rutinas.
- Tablas: rutinapesas

```
delimiter //
create procedure sp_registrarRutinaPesas(
in descrpicion text, in idCliente int, in idEntrenador int)
begin
insert into rutinaspesas(descripcion, id_cliente,id_entrenador) values(descripcion, idCliente, idEntreandor);
end;
```

```
57 •  call sp_registrarRutinaPesas("Rutina fuerza nivel 3", 1, 2);
58 •  select * from rutinapesas;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

id_rutina	descripcion	id_cliente	id_entrenador
1	Rutina fuerza nivel 1	1	2
2	Rutina resistencia nivel 2	2	2
3	Rutina fuerza nivel 3	1	2
4	Rutina fuerza nivel 3	1	2
NULL	NULL	NULL	NULL

8-SP registrar rutinas pesas

SP: sp_registrarPago

- Descripción: Inserta un nuevo pago de un cliente por una membresía.
- Objetivo: Facilitar la gestión de pagos.
- Tablas: pago

```
delimiter //
create procedure sp_registrarPago( in fecha date, in idCliente int, in idMembresia int)
begin
insert into pago(fecha, id_cliente, id_membresia) values(fecha, idCliente, idMembresia);
end;
```

```
call sp_registrarPago("2026-04-05", 6, 1);
select * from pago
```

Result Grid | Filter Rows: | Edit:

id_pago	fecha	id_cliente	id_membresia
1	2026-04-01	1	1
2	2026-04-02	2	2
12	2026-04-05	6	1
13	2026-04-06	1	1
NULL	NULL	NULL	NULL

9-SP registrar pago

Trigger

Trigger: trg_asistencia_auto

- Descripción: Al insertar un pago, se registra automáticamente una asistencia.
- Objetivo: Evitar carga manual de asistencias.
- Tablas: pago, asistencia

```
delimiter //
create trigger tgr_asistencia_auto after insert on pago for each row
begin
insert into asistencia(fecha, id_cliente, id_sesion) values (new.fecha, new.id_cliente,1); -- ejemplo: sesion fija
end;
```

```
insert into pago (fecha, id_cliente, id_membresia)
values ("2026-04-06", 1, 1);
select * from asistencia;
```

Result Grid | Filter Rows: | Edit: | Exp

id_asistencia	fecha	id_cliente	id_sesion
5	2026-04-03	1	1
6	2026-04-04	2	2
7	2026-04-05	6	1
8	2026-04-06	1	1
NULL	NULL	NULL	NULL

10-Trigger de asistencia auto

Conclusión

Con esta segunda entrega se completa la base de datos del gimnasio, incorporando vistas, funciones, procedimientos almacenados y un trigger opcional que amplían la funcionalidad del esquema inicial. Las pruebas realizadas confirman que los objetos creados funcionan correctamente y permiten gestionar clientes, entrenadores, membresías, pagos y asistencias de manera más eficiente. El proyecto cumple con los requisitos solicitados y deja preparada la base para futuras aplicaciones web o sistemas de administración digital.

link al repositorio de GITHUB: Julian-Iglesias/Base-de-datos-de-GIMNASIO